

PES UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE ENGINEERING LAB

Lab 3: Component Modelling & Architectural Pattern Selection

Student Name	Nishanth T Chigilar
SRN	PES1UG24CS302
Course	Software Engineering Lab (UE24CS252AA)
Assigned Problem Statement	Problem Statement #56 — Peer Skill Exchange & Mentorship Network

1. Executive Summary & System Overview

This document evaluates candidate architectural patterns and defines the component-level software architecture for Problem Statement #56: Peer Skill Exchange & Mentorship Network. The platform facilitates non-monetary skill exchange through time-banking, requiring real-time mentor discovery, atomic time credit escrow holds during session reservations, dual-confirmation settlements, and immutable transaction ledger logging.

2. Architectural Style Evaluation

Architectural Pattern	Structural Organization	Key Strengths / Pros	Architectural Limitations / Cons	Suitability for BPS #56
Layered (N-Tier) Architecture	Organized in horizontal, decoupled layers (Presentation, Business, Data).	Strong separation of concerns, simple deployment, strict data encapsulation, zero network IPC overhead.	Potential performance overhead if call chains pass through redundant intermediate layers.	High (Selected)
Microservices Architecture	Suite of small, independently deployable services communicating over network APIs.	High horizontal scalability, technology diversity, and independent service deployment.	Operational complexity, network latency, difficult two-phase commit (2PC) ACID ledger consistency.	Low (Excessive overhead)
Client-Server Architecture	Centralized server servicing multiple client interfaces.	Centralized control, straightforward session handling, simple data consistency.	Single point of failure; backend scaling bottlenecks during high concurrent session booking peaks.	Moderate

3. Architecture Selection Statement

“We chose Layered Architecture for the Peer Skill Exchange & Mentorship Network System.”

4. Scenario-Related Justification

1. Decoupling User Interactions from Time-Banking Ledger Rules: The platform handles distinct operational responsibilities: high-frequency search/filtering, session reservation holding, and credit ledger adjustments. Layered Architecture isolates user-facing components in the Presentation Layer from core credit accounting rules in the Business Layer. This guarantees that modifications to the frontend UI (e.g., mobile web layouts or search filter sliders) do not impact the underlying escrow calculations or ledger state logic.

2. Ensuring Strict Transactional Consistency Across Credit Escrows: Peer credit transfers require strict atomic transactions when locking 1 time credit into escrow during booking. Executing business logic within a unified Business Layer ensures ACID transaction compliance across the Session Booking & Escrow Component, Time-Banking Ledger Component, and Database Component without experiencing the network partition risks or eventual consistency issues inherent to microservice networks.

5. Security Advantage

Strict Isolation of Sensitive Credit Accounting & Encrypted Data Layers: Layered architecture enforces strict unidirectional dependency rules (Presentation → Business → Data). The User Interface Component residing in the Presentation Layer cannot directly access or query the Time-Banking Ledger Component or Database Component. All state modifications must pass through validated business interfaces (Search & Discovery API and Session Booking & Escrow API). This architectural boundary prevents client-side parameter tampering, unauthorized balance overrides, and direct SQL injection vectors.

6. Performance Benefit

Low-Latency Inter-Component Calls & In-Memory Availability Caching: Co-locating mentor searching, slot reservation, and credit ledger operations within the Business Layer enables high-frequency mentor profile and schedule queries to be served using in-memory caching directly inside the Mentor Discovery Component. Eliminating network RPC calls between business modules reduces end-to-end checkout latency to under 50 milliseconds, ensuring smooth user booking flows under high concurrent user traffic.

7. Component & Interface Specification

Component Name	Layer	Stereotype	Provided Interfaces (Ball)	Required Interfaces (Socket)	Component Description & Functional Scope
User Interface Component	Presentation Layer	<<component>>	None	Search & Discovery API; Session Booking & Escrow API	Renders client web/mobile dashboards, skill search forms, and reservation screens.
Mentor Discovery Component	Business Layer	<<component>>	Search & Discovery API	Database Interface	Processes multi-parameter searches, skill matching, mentor profile queries, and availability filters.
Session Booking & Escrow Component	Business Layer	<<component>>	Session Booking & Escrow API	Ledger Transaction API; Database Interface	Manages slot reservation state, verifies credit balance prerequisites, and holds 1 credit in temporary escrow.
Time-Banking Ledger Component	Business Layer	<<component>>	Ledger Transaction API	Database Interface	Executes non-monetary credit accounting, balance debits/credits, and immutable audit trail generation.
Database Component	Data Layer	<<component>>	Database Interface	Storage Engine / Disk	Enforces relational constraints, ACID transactional integrity, and persistent storage.

8. Component Diagram Visualization

Peer Skill Exchange & Mentorship Network (BPS #56) - Layered Architecture
Student: Nishanth T Chigilar | SRN: PES1UG24CS302

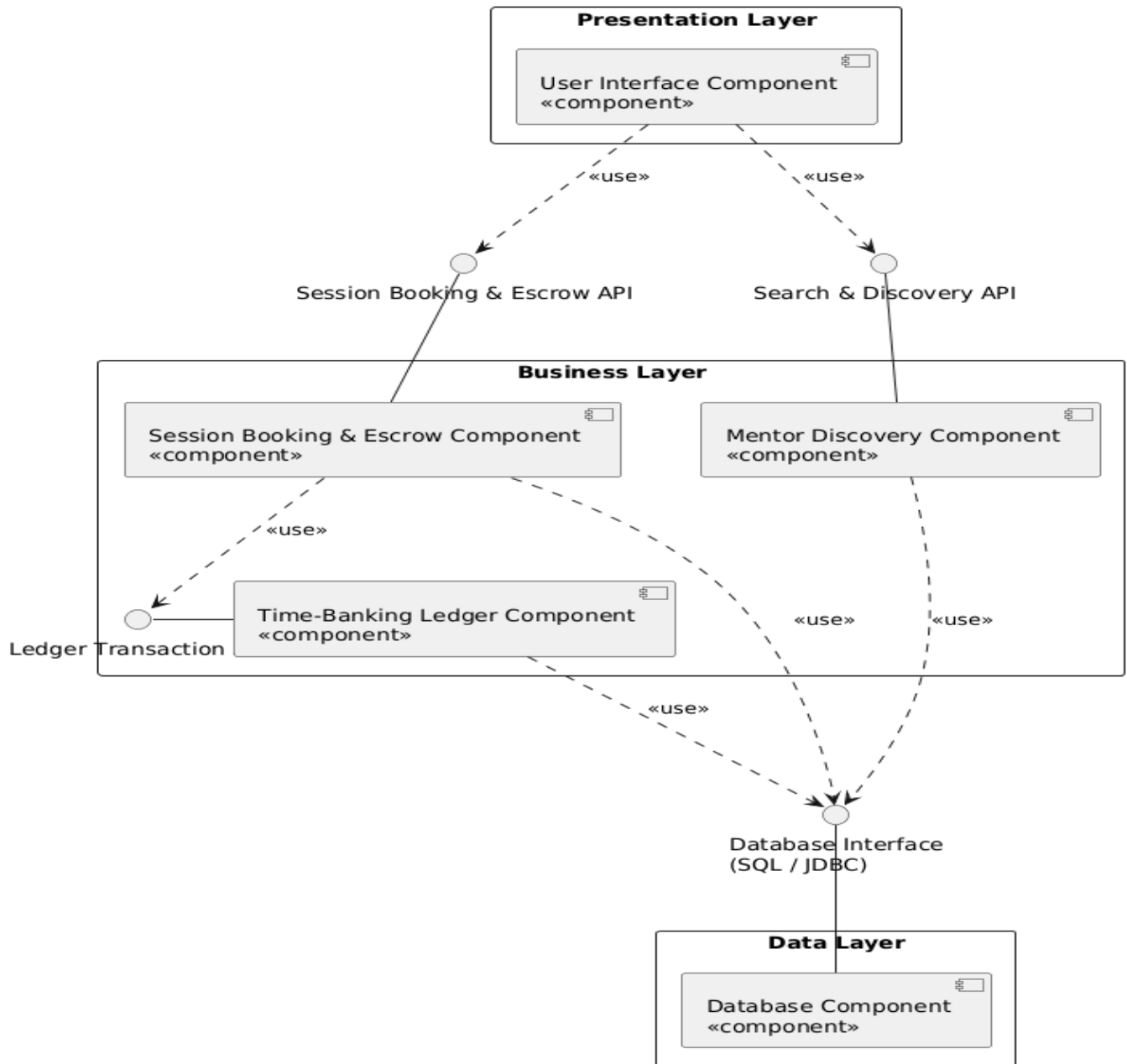


Figure 1: High-resolution UML Component Diagram showing the Layered Architecture, Provided/Required Interfaces, and Dependency Flow for Problem Statement #56.